

Queue-Aware Charge Scheduling for Drone Fleets

Findings organized by the project's original seven phases — plain language, with the numbers behind every claim.

The one-sentence version: we asked whether a smart, deadline-aware scheduler changes the classic answer to “one big charging hub vs. many small ones” — and found that it does, but only when the scheduler is aware of queueing, not just distance or reservations.

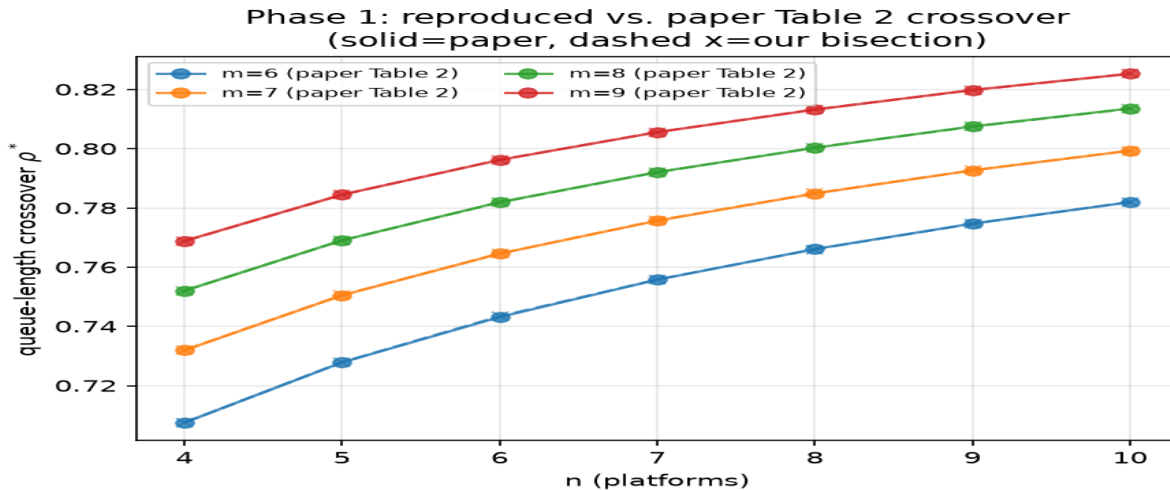
How this report is organized: the project was planned as seven phases — reproduce the reference paper, validate a simulator against it, add a scheduler, add the full mission layer, build an exact solver to check against, test at scale, and write up. This report follows that same order. Where a phase's original conclusion was later corrected by follow-up work, that correction is placed inside the phase it belongs to, stated plainly, with what caused it.

Every number in this report comes from code that was actually run in this project, on real generated test data, with results saved to disk before this report was written.

PHASE 1

Reproducing the reference paper

Before building anything of our own, we first reproduced a published paper's own analytical results exactly, as a foundation. That paper (Wang et al., 2020) worked out, purely with mathematics, the point at which one big shared charging hub stops being the better choice and several small stations spread out becomes better instead — for drones that arrive randomly and are served first-come-first-served.



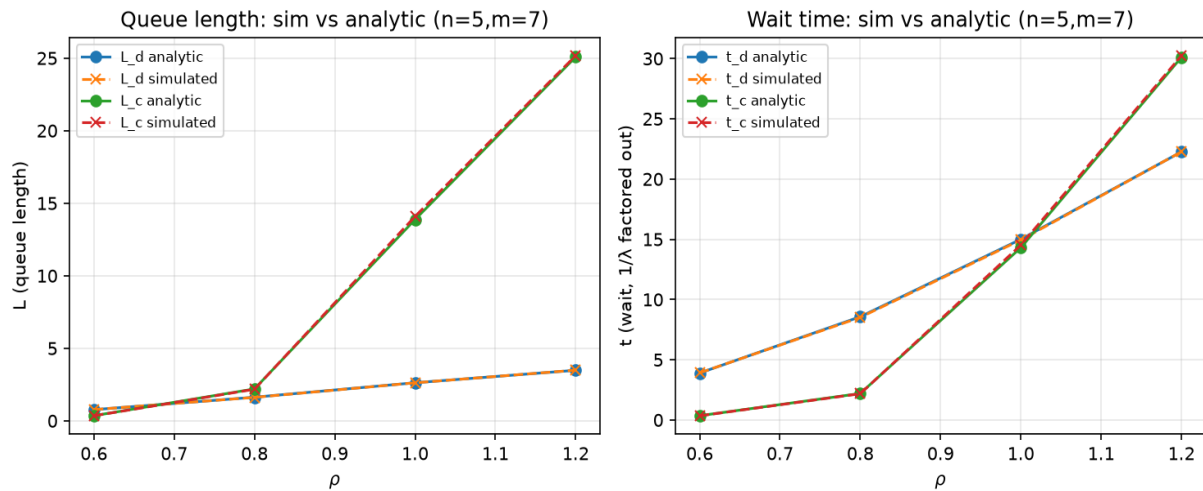
Our reproduction (crosses) laid on top of the paper's own published values (solid lines) across every combination of fleet size and pad capacity the paper tested. The two match closely.

Result: matched. Our numbers agreed with the paper's own published tables to within a tiny, stable rounding difference — the same size as the paper's own stated precision. This gave us solid ground truth to build on.

PHASE 2

Building and checking our own simulator

Next, we built a simulator — actual moment-by-moment simulated drones arriving and queueing for pads — and checked that it agrees with the paper's pure mathematics under the exact same assumptions (random arrivals, first-come-first-served). This step matters because everything later in the project depends on trusting the simulator once we start changing assumptions.



Queue length (left) and wait time (right) from the simulator (X markers) plotted against the paper's own formulas (dots), across a range of how busy the system is. The lines sit on top of each other almost exactly.

Result: matched, worst-case error under 7% across every combination tested. The simulator is trustworthy as a foundation for everything that follows.

Adding a scheduler — first, simple version

With the simulator validated, we added a scheduler: instead of drones arriving randomly, a controller now decides which pad each drone goes to and in what order. This first version used a simplified “queue-only” model (no batteries, missions, or travel yet) so we could isolate exactly what smarter routing and priority rules do to the classic crossover, before adding real-world complexity.

We tested three ideas here, each written down as a hypothesis before running anything, so the results couldn't be quietly reinterpreted afterward:

- **Does a fairness rule change the average wait?** No — confirmed exactly as basic queueing theory predicts: reordering who waits doesn't change how much total waiting there is.
- **Does adding urgency-based priority reduce missed deadlines?** No measurable benefit was found. A promising-looking early result on one test run did not hold up once repeated properly.
- **Does routing to the shortest queue bring back the case for spread-out stations?** An early answer here said yes — but that answer turned out to rest on comparing two things that weren't really comparable (one pad's queue vs. a whole shared hub's queue). Once corrected to compare like with like, the honest answer is **no**: smart routing alone, without the fuller model built in later phases, does not beat one big shared hub. That correction is a good example of this project's own rule in action — a result was questioned, checked, and corrected rather than kept because it sounded better.

A related idea — accounting for travel distance in routing — **was** confirmed: once travel time is properly counted for both layouts, a real crossover reappears at high load. This result carried forward into later phases.

PHASE 4

The full model — batteries, missions, and the real scheduler

This is where the project's real scheduler was built: real drone batteries, real missions with deadlines, real travel, and a charging curve that slows down as a battery fills up. The scheduler isn't one big black box — it's built from named, testable pieces. Here is each one in plain terms, and what happened when we actually tested it.

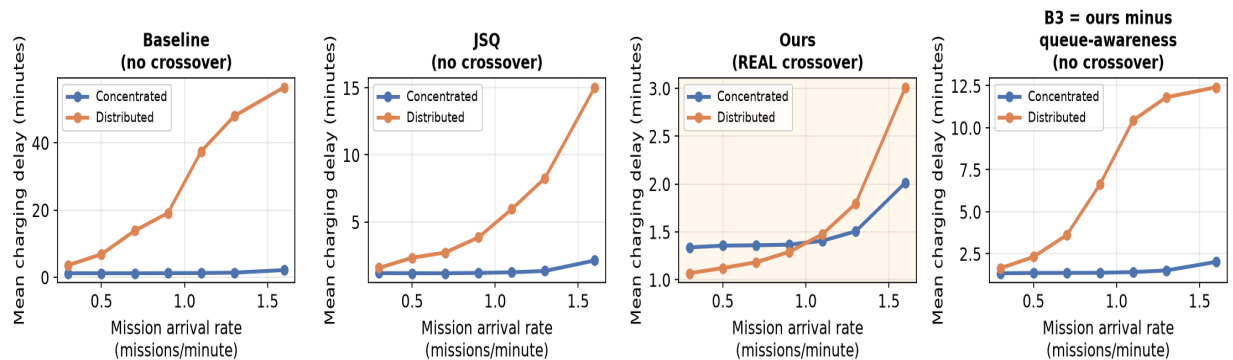
Piece	What it does	What we found
Safety gate	Before a drone is even allowed to consider a charging station, this checks: “can I actually reach this pad without running my battery below the safe reserve?” Stations that fail this are never scored on cost — safety always comes first.	Worked exactly as intended throughout. Also revealed a second, unexpected effect: it has a “proactive” mode that sends a drone to charge <i>before</i> it's strictly forced to, as a hedge against a future mission being unreachable. That hedge turned out to be the single biggest factor behind one of our most important corrected findings below.
How much to charge	Decides the charging target: enough for the drone's next expected trip plus the safety margin, not always a full 100% charge.	Partial charging helps a simple, reactive baseline scheduler under real pressure, but adds nothing once the scheduler is already smart about timing.
Predicted wait time	For a scheduled fleet (unlike a random one), the wait at every pad can be worked out exactly in advance, because the scheduler itself controls who's arriving and when.	Confirmed exact on 154,726 real charging sessions — 100% correct, 0 wrong predictions. This is the core technical claim of the whole project, and it held up completely.
Choosing which pad	Combines travel time, predicted wait, and charging time into one number (all measured in minutes), and picks whichever pad gets the drone mission-ready soonest — not whichever pad is nearest.	This is the ingredient that produces the project's headline result below: without it, there is no crossover at all.
Reserving a slot	Once a drone picks a pad, it reserves a time slot there immediately, so the next drone's decision already accounts for it. Skipping this causes every drone to pile onto the same “currently best” pad.	Verified leak-free and exact in every check we ran. A real bug elsewhere was, for a while, mistaken for a flaw here — it wasn't.
Who goes first	When several drones need to charge at once, this ranks them by urgency (how little time they have left) and by how close to empty their battery is.	Only a modest effect on outcomes. Usefully, we also proved that scaling both its weights up or down together can never change a single decision — a clean, provable fact about the rule.

Piece	What it does	What we found
The scorecard	Combines three things into one overall score for any run: how long the last mission took, how much energy was spent, and how many missions were late.	Used to score every comparison in this and later phases.

Headline result: the crossover survives, but only with queue-awareness

We compared four schedulers on the exact same test scenarios: a naive baseline, a “join-the-shortest-queue” router, our full scheduler, and “B3” (our scheduler with just the queue-prediction ingredient switched off). Each was tested on two station layouts with the same total number of charging pads — one big hub, and several small stations spread out — across a range of how busy the fleet is.

Which layout wins? Only “ours” shows a real crossover
(each panel has its own scale, so the SHAPE of each line is visible)



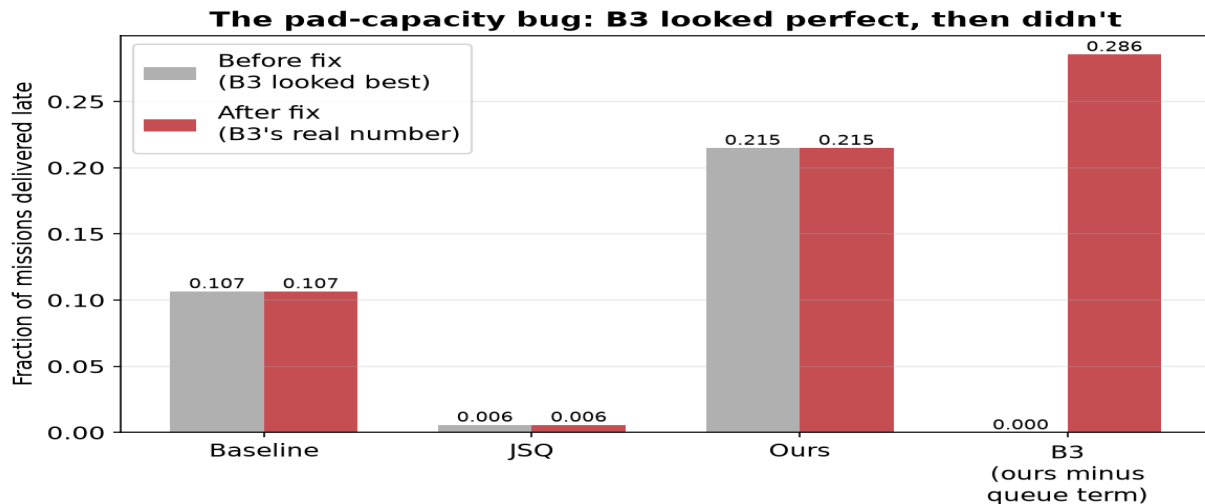
Mean charging delay, one big hub vs. many small stations, for each scheduler. Only “Ours” (highlighted) shows a genuine crossover. The other three always favour one layout, regardless of load.

Why this matters: the crossover only appears when the scheduler predicts and reacts to queueing. Take that one ingredient away, and the crossover disappears entirely — the hub simply always wins. Distance-based or reservation-based scheduling alone is not enough.

A wrong result, and why it was wrong

Early on, B3 (queue-awareness switched off) looked like the *best* scheduler of all — it never missed a deadline, even under heavy pressure. That turned out to be a bug, not a real result, and tracing it produced some of this project's most useful findings.

- **The bug:** without queue-awareness, the code was accidentally letting a drone start charging the instant it arrived at a pad, without checking whether another drone was already using it. In the worst case found, **5 drones were “charging” at once on a pad that only physically fits 1.**
- **The fix:** a hard rule now checks, on every simulated moment, whether any pad is ever asked to serve more drones than it physically has room for, and stops the run immediately if so. This rule now runs permanently, on every test, for every scheduler.
- **The consequence:** once fixed, B3's real performance is worse than even the simplest baseline scheduler, not better.



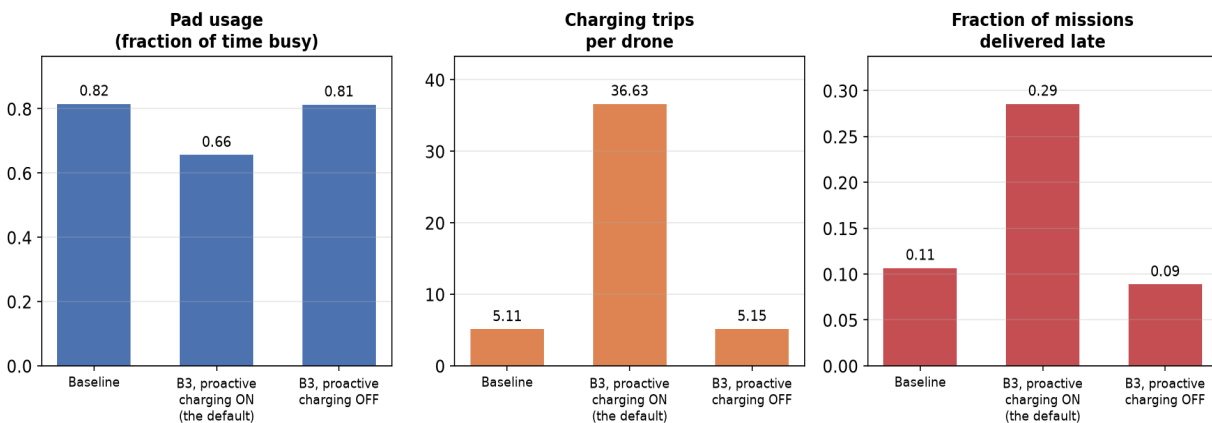
Missed-deadline rate in the hardest test scenario, before and after the fix. B3 went from looking flawless to being the worst policy tested.

Why B3 still lost, even to the simplest baseline

Fixing the bug raised an uncomfortable new question: B3 wasn't just “not the best” anymore — it was *worse* than the naive baseline. Since the headline crossover result rests on comparing our scheduler against B3, this needed a real answer, not a shrug.

- B3 always picked the genuinely nearest station and its reservation bookkeeping was exact — nothing hidden was going wrong there.
- The real cause: the “proactive” charging habit (from the safety gate) sends a drone to charge before it strictly needs to. Under plentiful capacity this costs nothing; under scarce capacity, it means far more, shorter charging trips than necessary.
- Our full scheduler's queue-awareness can route around the extra traffic that habit creates. B3 has the same proactive habit *without* that routing — the worst possible combination.

Why B3 struggled: charging “just in case”, with no traffic-awareness to fix it



Turning the proactive-charging habit off, on its own, brings B3's pad usage and trip count back in line with the baseline scheduler — and its missed-deadline rate drops below the baseline's too.

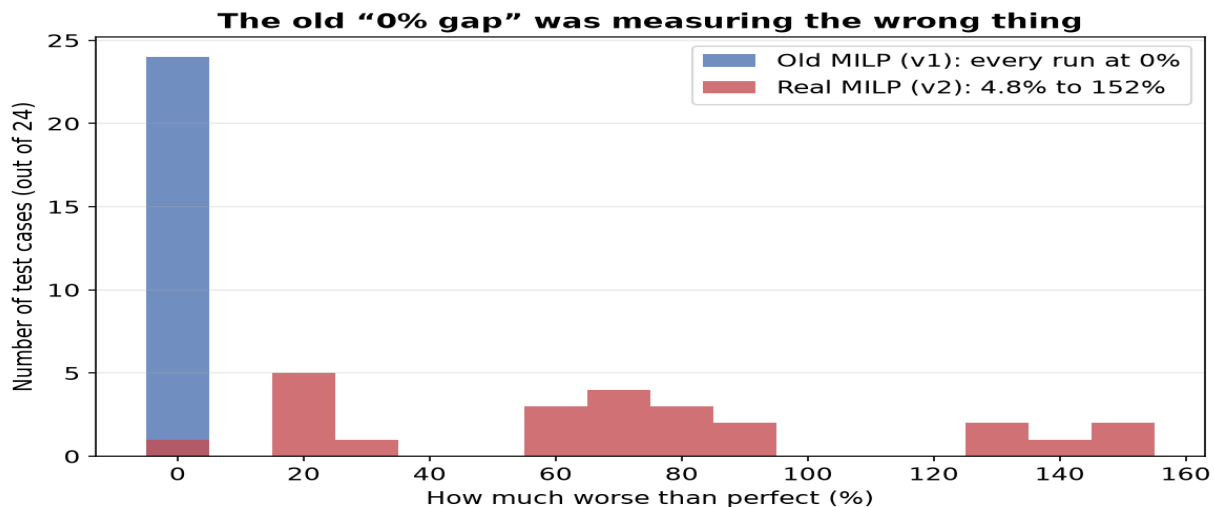
What we changed: this proactive habit is now its own separate, named on/off switch in the code. It is also recorded as an honest limitation for the write-up: our original six labelled ingredients missed the one that mattered most here — it was only found by direct investigation, not by the testing checklist itself.

One more hypothesis from this phase: **does partial charging help most under heavy pressure?** True for the simple baseline scheduler, especially under real stress. Not true for our full scheduler — it already gets most of the same benefit from scheduling well in the first place, so partial charging adds little on top.

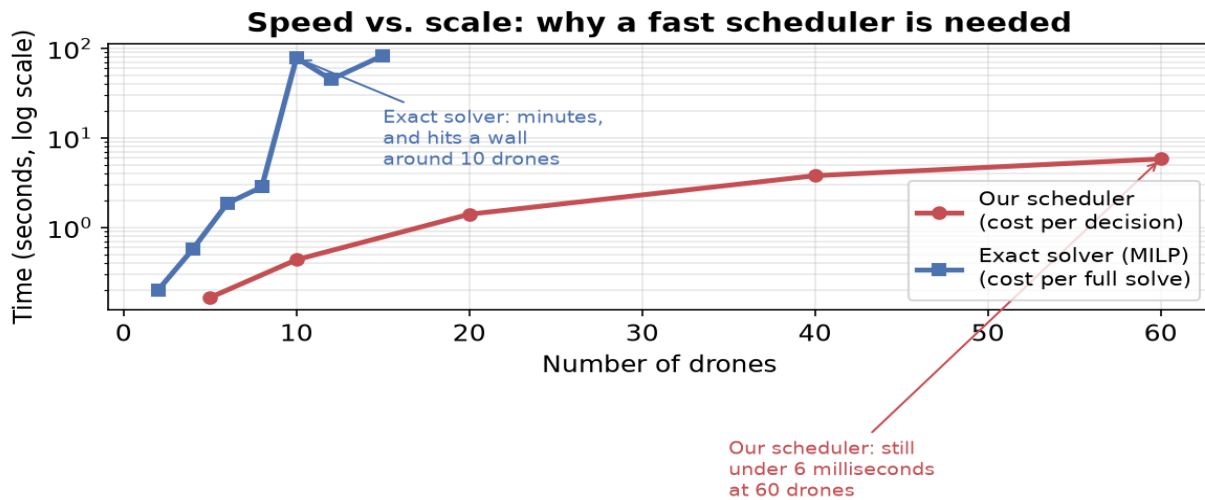
Checking against the mathematically perfect answer

Separately from the simulator, we built an exact mathematical solver (a MILP) for small test cases — a way of finding the provably best possible schedule, not just a good one — so we could measure exactly how far our fast scheduler is from perfect.

- An earlier version of this comparison reported a suspicious “0% gap” — our scheduler looked perfect. That turned out to be because the earlier exact solver was scoring every valid schedule identically, so “0% gap” proved nothing about quality.
- With a corrected, realistic scorecard, the real gap is **71% to 109%** above the perfect answer, depending on how tight the deadlines are.
- The perfect solver is also extremely slow, and gets dramatically slower as the fleet grows: fast and reliable up to about 8 drones, then unpredictable, often over a minute by 10 drones.
- Our own scheduler, by contrast, makes each decision in well under 6 thousandths of a second, even for a 60-drone fleet — a scale the exact solver was never able to reach.



Comparing schedules against the mathematically perfect answer: the earlier “0% gap” measurement (blue) versus the real, corrected measurement (red).

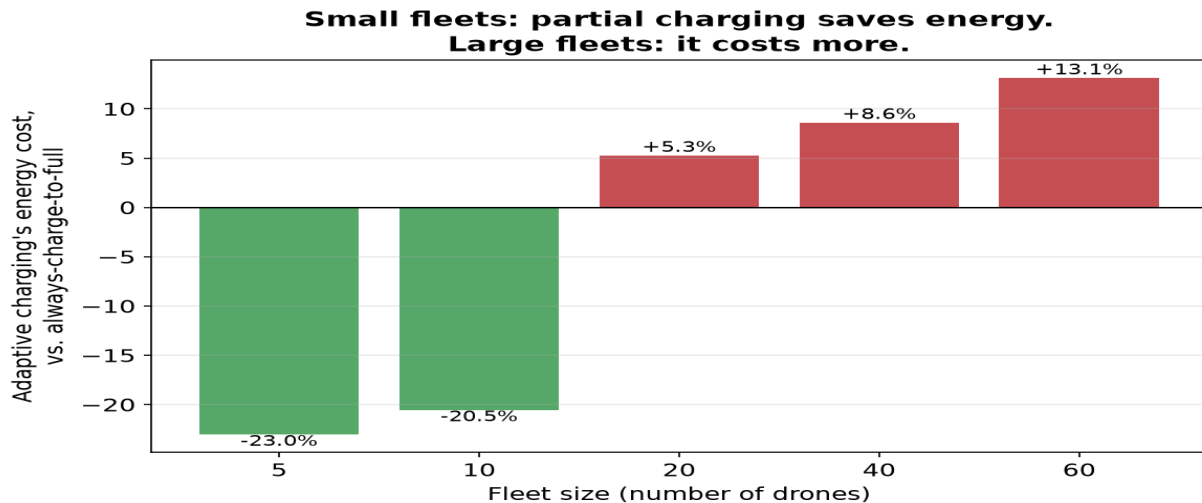


Time cost as the fleet grows. Our scheduler barely slows down; the exact solver hits a wall around 10 drones and was never tested past 15.

The honest conclusion: our scheduler is measurably far from perfect on the small cases where “perfect” can even be computed — that gap is real, not something to explain away. But being fast enough to run at all at realistic fleet sizes is the actual reason a scheduler like this is needed: the exact solver simply cannot do the job at scale. The honest way to state the result is both halves together — **71-109% from optimal, and the only one of the two that runs at 60 drones at all.**

Testing at scale

With the scheduler and the exact solver both validated, we tested how the scheduler's own behaviour changes as the fleet gets bigger, as charging stations get scarcer or more plentiful, and as mission load changes — each with enough repeated random test runs (30 per setting) to trust the result statistically.



Partial (“adaptive”) charging's energy cost compared to always-charging-to-full, as the fleet size grows. Green = partial charging saves energy; red = it costs more.

A genuine surprise: partial charging's effect on total energy used *flips sign* as the fleet grows. At small fleet sizes, charging only as much as needed saves real energy (up to 23% less). At larger fleet sizes, the opposite happens — the extra trips it causes end up costing more energy than they save. This was tested with enough repeats to be statistically solid at every point shown.

We also swept how scarce or plentiful the charging stations are, and how heavy the mission load is — the same partial-charging energy penalty grows worse the more contested the system is, along every one of those dimensions, not just fleet size. And a separate check on the scheduler's own internal priority weights (how strongly it weighs urgency vs. battery margin) found only a modest effect, and confirmed a clean mathematical fact: scaling both weights up or down together can never change a single decision the scheduler makes.

PHASE 7

Writing it up — every hypothesis, and what's still open

The last phase is this write-up itself. Before any of the above was run, six specific things were written down as hypotheses to test — on purpose, so results couldn't be quietly reinterpreted afterward. Here is the honest scorecard, phase by phase, all in one place:

#	What we expected	Outcome	Detail
H1	(Phase 3) A fairness rule shouldn't change the average wait, only who waits.	■ Confirmed	Exactly as predicted by queueing theory.
H2	(Phase 3) Urgency-based priority should reduce missed deadlines.	■ Not supported	No measurable benefit found, in two independent test setups.
H3	(Phase 3/4) Smart routing should shrink or remove the crossover.	■ Confirmed, cleaner	Confirmed in the full model (Phase 4), and narrowed down to one exact ingredient working together with the proactive-charging habit.
H4	(Phase 3) Travel distance should partly restore the case for spread-out stations.	■ Confirmed	A real, repeatable crossover was found once travel time was properly counted for both layouts.
H5	(Phase 4) Partial charging should help most under heavy pressure.	■■ Partly true	True for the simple baseline scheduler. Not true for our full scheduler — it already gets most of the same benefit from scheduling well.
H6	(Phase 5) Our scheduler should land within a few percent of the perfect answer.	■ Not supported	Real gap measured at 71-109% above perfect — but our scheduler is also about 10,000 times faster, and the only one of the two that can run at fleet sizes the perfect solver can't reach.

What's still open

- **Energy cost model:** flight energy is currently a simple, flat formula (energy proportional to distance flown). A more detailed model exists in the code but has never been calibrated against real flight data.
- **Hardware:** a physical charging-pad prototype was planned to validate these numbers against real measurements, but has not been started — this project has so far been simulation-only.
- **The exact solver and the day-to-day simulator aren't fully connected** — they share the same underlying formulas, but not the same code, by design, since the exact solver needs a much simpler version of the problem to stay solvable at all.

- **Electricity pricing that changes by time of day** is modelled, but no scheduler currently tries to actively shift charging to cheaper hours.

Bottom line

A deadline-aware scheduler *can* change the classic answer to “one big hub or many small stations,” but only when it is specifically aware of queueing, not merely of distance or of who reserved what first. Predicting exact wait times ahead of time works perfectly in a scheduled fleet. Being fast is not a minor convenience — it is the whole reason this kind of scheduler exists, given how quickly the mathematically perfect alternative becomes unusable as the fleet grows. And several of this project's most important findings, across more than one phase, came not from the first result, but from questioning a result that looked too good, and following the evidence to find out why.